



Introduction: your first Phaser 4 game

Overview

The first contact with Phaser, Deno and TypeScript. By the end, students have built and run two projects, can find their way round a project folder, and can read (and make small changes to) a config, a scene, and a game object class. Nothing here is hard; the risk in this chapter is **tooling friction** and **TypeScript surprises** for Java programmers, so leave time for both.

Prerequisites

- comfortable writing classes, constructors, fields and methods in Java (or C#)
 - has seen a little JavaScript or HTML - not essential
 - Deno 2.4 or later installed (`deno --version`); Celbridge installed, or any editor plus a terminal
 - a network connection for the **first** build of the first project (Phaser is downloaded once, into Deno's cache, and shared by every project after that)
-

Learning outcomes

Students can:

1. build and serve a Phaser project, and explain why it has to be served rather than opened as a file
2. identify the config, the scenes and the game objects in a project, and say what each is for
3. read and write the TypeScript used in the chapter: typed fields and parameters, `const / let` , classes with `extends / super / override` , `this.` , arrow functions, template strings, `import / export`
4. place game objects using the coordinate system and `setOrigin`

5. describe the game loop, and use `delta` to move something at a speed in pixels per second

Suggested session plan (2 x 1 hour, or one 2 hour lab)

Time	Activity
0:00 - 0:15	Slides 1-6: what Phaser is, the tools, the build pipeline. Live demo: open <code>ch01_hello Phaser</code> in Celbridge, build, serve, open the webview
0:15 - 0:35	Students do the same. Circulate - this is where tooling problems surface (see below)
0:35 - 0:55	Slides 7-11: TypeScript for Java programmers; config, scene, game objects. Walk through <code>HelloScene.ts</code> line by line
0:55 - 1:10	Challenges 1-3
1:10 - 1:25	Slides 12-15: coordinates and origins, the game loop, <code>delta</code> . Live demo: change <code>speedX</code> in <code>ch01_moving_ball</code> ; then change the movement to "4 pixels a frame" and discuss why that is wrong
1:25 - 1:55	Challenges 4-6 (6 is a stretch; fine to set as homework)
1:55 - 2:00	Recap: the three parts of a game; the loop

Key points to stress

- **The game is always 800 x 600 inside**, whatever size it appears. Students who make the browser window small and then try to "fix" coordinates have misunderstood the scale manager
- **You never draw.** You add game objects and change them; Phaser draws them every frame. Java students who have used `paint(Graphics g)` expect to redraw things themselves
- `this.` **is compulsory** inside a class. Forgetting it is the single most common error this week

- **Speeds are per second, and `delta` is in milliseconds.** Get this right now and Chapter 4 and Chapter 8 are much easier

Common problems and errors

What students see	Cause	Fix
Blank page, or the page loads with no game	opened <code>dist/index.html</code> as a file, or the server is not running	run <code>deno task serve</code> , and open http://127.0.0.1:8000/
<code>address already in use</code> when serving	another project's server is still running (often in another terminal tab)	stop it with Ctrl+C, or just use it - it serves whichever <code>dist/</code> it was started in, so stop it before switching project
Old version of the game after changing the code	forgot to rebuild, or the browser cached the page	build, then refresh (Ctrl+Shift+R forces it). Suggest <code>deno task dev</code>
Cannot find name 'message'	wrote <code>message</code> instead of <code>this.message</code>	fields always need <code>this.</code>
This member must have an 'override' modifier	wrote <code>update(...)</code> in a scene without <code>override</code>	add <code>override</code> . Worth explaining: the compiler is checking that the method really replaces one
This member cannot have an 'override' modifier because it is not declared in the base class 'Image'. Did you mean 'update'?	copied <code>override</code> onto <code>Ball.preUpdate</code>	<code>Image</code> has no <code>preUpdate()</code> ; remove <code>override</code> (Chapter 4 explains when it is needed)
Property 'message' has no initializer and is not definitely assigned in the constructor	left the <code>!</code> off a field set in <code>create()</code>	add the <code>!</code> , and explain what it promises
Module not found <code>"file:///.../HelloScene"</code>	import without <code>.ts</code>	Deno needs the full file name
A picture shows as a green-and-black box, and the console says <code>Failed to process file: image "logo"</code>	wrong path in <code>this.load.image</code> , or the file was not copied into <code>public/assets/</code>	paths are relative to <code>index.html</code> ; check <code>public/assets/images/</code>

What students see	Cause	Fix
Keys do nothing	the game does not have keyboard focus	click the game first

Discussion questions

- Why does the build still produce a game when there are type errors? When is that helpful, and when is it dangerous?
- The ball moves itself in `preUpdate()` ; the scene could move it in `update()` instead. What is gained by putting the code in `Ball` ? (Encapsulation; many balls for free - challenge 4.)
- What would happen to a game written as "move 4 pixels a frame" on a 144 Hz monitor? On a slow school PC that manages 30 frames a second?

Extension ideas

- Look at `game.loop.actualFps` in the Moving Ball project with the browser's developer tools performance throttling turned on: the ball's speed should not change
- Read `build.ts` together - it is short, and shows how Deno runs other commands
- Try `type: Phaser.CANVAS` in the config and compare; discuss why WebGL is the default

Assessment ideas

- A short practical: "make a scene with your name centred at the top, a picture in the bottom right corner using `setOrigin` , and a key that changes the text"
- Code reading: give students `Ball.ts` with `this.` removed in three places and the `!` missing from a field, and ask them to predict the compiler's messages
- Explain, in two or three sentences, why speeds are multiplied by `delta / 1000`

Challenge solutions

See [solutions.md](#). Every challenge has a runnable solution in `solutions/`.

Solutions

Each solution is a complete project in `solutions/`, made from the chapter project the challenge starts from. Every change is marked with a `// CHALLENGE n` comment, so a search for `CHALLENGE` finds them all.

1. Make it yours

Project: `solutions/ch01_challenge_1_make_it_yours` (from `ch01_hello_phaser`)

Goal: get comfortable finding and changing values: the config, the page, the text style, an array.

`src/main.ts`

```
title: "Star Painter",  
...  
backgroundColor: "#264653",
```

`src/scenes/HelloScene.ts`

```
const COLOURS = ["#1d2433", "#3a0ca3", "#2a9d8f", "#9d0208", "#264653",  
"#6a4c93", "#ff006e", "#fb8500"];  
...  
this.message = this.add.text(centreX, 380, "Press SPACE", {  
  fontFamily: "Arial",  
  fontSize: "48px",  
  color: "#ffd166",  
});
```

and `<title>Star Painter</title>` in `public/index.html`.

Look for: the title changed in **both** places (the config's title and the page's `<title>` are separate things). `fontSize` accepts either a number (pixels) or a CSS string such as `"48px"`; the guide uses strings because the style object follows CSS - either is correct.

2. Corner labels

Project: `solutions/ch01_challenge_2_corner_labels` (from `ch01_hello_phaser`)

Goal: use origins to line things up against edges without measuring text.

`src/scenes/HelloScene.ts`

```
const w = this.scale.width;
const h = this.scale.height;
this.addLabel(36, 8, "(0, 0)", 0, 0);
this.addLabel(w - 36, 8, `(${w}, 0)`, 1, 0);
this.addLabel(36, h - 8, `(0, ${h})`, 0, 1);
this.addLabel(w - 36, h - 8, `(${w}, ${h})`, 1, 1);
```

```
private addLabel(x: number, y: number, label: string, originX: number,
originY: number): void {
  const text = this.add.text(x, y, label, {
    fontFamily: "Arial",
    fontSize: "16px",
    color: "#ffd166",
  });
  text.setOrigin(originX, originY);
}
```

The right-hand labels use origin `x = 1`, so `x` is their **right** edge and they grow leftwards from it; the bottom labels use origin `y = 1`. The labels are 36 pixels in from the corner so they clear the 32 pixel stars.

Look for: no `text.width` arithmetic; the corner values taken from `this.scale` rather than typed in as 800 and 600; a helper method rather than four copies of the style object (not required, but worth praising). Weaker answers hard-code `x = 720` and so on - it works, until the text changes.

3. Reset key

Project: `solutions/ch01_challenge_3_reset_key` (from `ch01_hello_phaser`)

Goal: a second event listener, and keeping "the starting state" in one place.

```
const START_COLOUR = "#1d2433";  
const START_MESSAGE = "Press SPACE";
```

```
this.input.keyboard!.on("keydown-R", () => {  
  this.reset();  
});
```

```
private reset(): void {  
  this.presses = 0;  
  this.cameras.main.setBackgroundColor(START_COLOUR);  
  this.message.setText(START_MESSAGE);  
}
```

The same message is now used in `create()` and in `reset()`, so it became a constant.

Look for: constants for the starting values (the "same thing written twice" smell); the reset in a named method. A common mistake is setting `this.presses = 0` but not the text, so the screen still says "Presses: 7" until the next press.

4. More balls

Project: `solutions/ch01_challenge_4_more_balls` (from `ch01_moving_ball`)

Goal: see the pay-off of a class that looks after itself: many balls, no extra moving code.

`src/scenes/GameScene.ts`

```

const BALL_COUNT = 5;
const MAX_SPEED = 300;
...
for (let i = 0; i < BALL_COUNT; i++) {
  const x = Phaser.Math.Between(32, this.scale.width - 32);
  const y = Phaser.Math.Between(32, this.scale.height - 32);
  const speedX = Phaser.Math.Between(-MAX_SPEED, MAX_SPEED);
  const speedY = Phaser.Math.Between(-MAX_SPEED, MAX_SPEED);
  new Ball(this, x, y, speedX, speedY);
}

```

`Ball.ts` does not change at all - which is the point to draw out.

Look for: start positions that keep the whole ball on screen (a ball that starts half off the edge gets stuck flipping its speed every frame until `Clamp` rescues it - a nice bug to discuss if someone hits it). A speed of 0 in one direction is possible and harmless.

5. Bounce counter

Project: `solutions/ch01_challenge_5_bounce_counter` (from `ch01_moving_ball`)

Goal: an object that keeps its own state private, and a scene that asks for it.

`src/objects/Ball.ts`

```

private bounces = 0;
...
if (this.x < radius || this.x > right) {
  this.speedX = -this.speedX;
  this.x = Phaser.Math.Clamp(this.x, radius, right);
  this.bounces = this.bounces + 1;
}
...
public getBounces(): number {
  return this.bounces;
}

```

`src/scenes/GameScene.ts`

```
private ball!: Ball;
private bounceText!: Phaser.GameObjects.Text;
...
this.ball = new Ball(this, 400, 300, 240, 180);
...
override update(time: number, delta: number): void {
    this.bounceText.setText(`Bounces: ${this.ball.getBounces()}`);
}
```

The scene now keeps the ball in a field - it needs to ask it something.

Look for: the count kept **in the ball** (it is the ball's fact) and read with a getter, rather than a public field the scene could change. Hitting a corner counts as two bounces in this solution (one each way) - accept either answer if the student can explain their choice. An alternative some students find: the ball emits an event when it bounces (Chapter 6 covers events properly).

6. Steer the ball

Project: `solutions/ch01_challenge_6_steer_the_ball` (from `ch01_moving_ball`)

Goal: polling keys every frame, and applying a change **per second** using `delta` - a force, in effect.

`src/objects/Ball.ts`

```
const TOP_SPEED = 600;
...
public push(amountX: number, amountY: number): void {
    this.speedX = Phaser.Math.Clamp(this.speedX + amountX, -TOP_SPEED,
TOP_SPEED);
    this.speedY = Phaser.Math.Clamp(this.speedY + amountY, -TOP_SPEED,
TOP_SPEED);
}

public getSpeed(): number {
    return Math.sqrt(this.speedX * this.speedX + this.speedY * this.speedY);
}
```

src/scenes/GameScene.ts

```
const PUSH_STRENGTH = 400;
...
this.cursors = this.input.keyboard!.createCursorKeys();
...
const push = PUSH_STRENGTH * (delta / 1000);
let pushX = 0;
let pushY = 0;
if (this.cursors.left.isDown) pushX = pushX - push;
if (this.cursors.right.isDown) pushX = pushX + push;
if (this.cursors.up.isDown) pushY = pushY - push;
if (this.cursors.down.isDown) pushY = pushY + push;
this.ball.push(pushX, pushY);
```

Holding RIGHT for one second from rest gives a speed of about 400 across - tested with the harness (399.3 after 1000 ms).

Look for:

- the push scaled by `delta` . Without it, the ball speeds up faster on faster screens - exactly the mistake the chapter warned about, one level up (this is **acceleration**, so it needs time as much as movement does)
- `createCursorKeys()` and `isDown` (polling) rather than `keydown` events, which fire once per press (and then repeat at the operating system's key-repeat rate - an interesting bug when tried)
- the limit applied in the ball, which owns the speed
- the overall speed shown with Pythagoras is a nice touch but not required; showing `speedX` and `speedY` is fine

Stronger students may notice this is what Arcade Physics' `setAcceleration` and `setMaxVelocity` do (Chapter 9).

Slides

Chapter 1

Introduction: your first Phaser 4 game

Phaser 4 2D games, in TypeScript, with Deno



1 / 19

Today

- what Phaser is - and what changed in Phaser 4
- building and running a project with Deno and Celbridge
- TypeScript, for Java programmers
- config, scenes, game objects
- coordinates and origins
- the game loop, and pixels per **second**

2 / 19

What is Phaser?

- a free, open source framework for **2D games in the browser**
- you write the rules; Phaser draws, loops, loads, reads input, plays sound, runs physics
- **Phaser 4** (2025): a new drawing engine; the API you write is almost the same as Phaser 3
- most tutorials online are Phaser 3 - nearly always still right
- ships with TypeScript types: the compiler knows every Phaser class and method

3 / 19

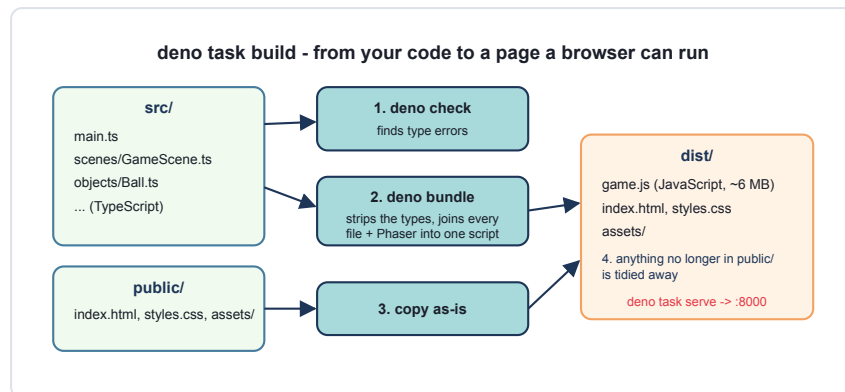
The tools

- **Deno** - runs TypeScript, type checks, bundles, serves. Nothing else to install
- the only package: **Phaser** (downloaded once, on the first build)
- **Celbridge** - `terminal.console` (buttons for each command) and `game.webview`

```
deno task build    type check + bundle into dist/  
deno task serve    http://127.0.0.1:8000/  
deno task dev      rebuild on every save
```

4 / 19

What "build" does



The game must be **served**: browsers will not load pictures for a page opened from a file.

5 / 19

TypeScript for Java programmers (1)

```
const SPEED = 200;           // never changes
let score = 0;                // changes - type worked out: number
let name: string = "Ada";     // type AFTER the name

7 / 2                          // 3.5 - there is only `number`
a === b                        // always ===, never ==

`Presses: ${this.pressures}` // template string
```

6 / 19

TypeScript for Java programmers (2)

```
export class GameScene extends Phaser.Scene {  
  private score: number;  
  
  constructor() {  
    super("GameScene");           // before using `this`  
    this.score = 0;               // ALWAYS `this.`  
  }  
  
  override update(_time: number, delta: number): void {  
    this.score = this.score + delta / 1000;  
  }  
}
```

`override` is compulsory here. `_time` = "not used".

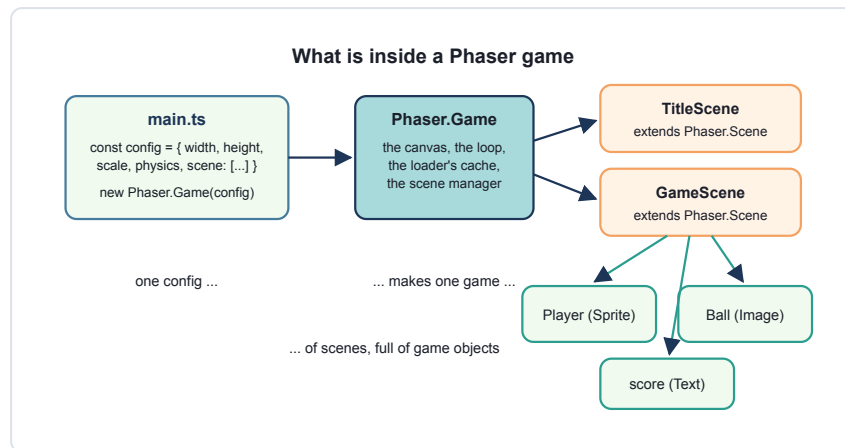
7 / 19

TypeScript for Java programmers (3)

```
private message!: Phaser.GameObjects.Text; // "set later, in create()"  
  
this.input.keyboard!.on("keydown-SPACE", () => { // arrow function:  
  this.changeColour();                          // `this` is still the scene  
});  
  
import { HelloScene } from "../scenes/HelloScene.ts"; // .ts on your own files
```

8 / 19

Inside a Phaser game



9 / 19

1. The config - main.ts

```
const config: Phaser.Types.Core.GameConfig = {
  type: Phaser.AUTO,
  parent: "game",
  backgroundColor: "#1d2433",
  scale: { width: 800, height: 600,
    mode: Phaser.Scale.FIT, autoCenter: Phaser.Scale.CENTER_BOTH },
  scene: [HelloScene],
};
new Phaser.Game(config);
```

Always **800 x 600 inside**, scaled to fit the page.

10 / 19

2. Scenes

Method	Called	For
<code>constructor()</code>	once	<code>super("HelloScene")</code> - its key
<code>preload()</code>	scene starts	load pictures and sounds
<code>create()</code>	all loaded	make game objects
<code>update(time, delta)</code>	every frame	the rules

11 / 19

3. Game objects

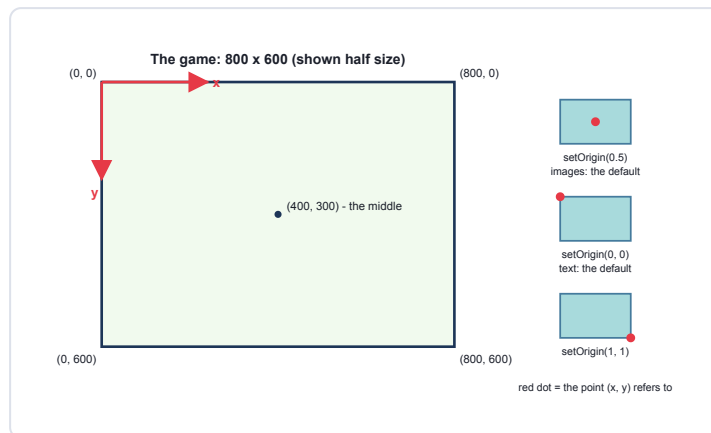
```
preload(): void {
    this.load.image("logo", "assets/images/logo.png");
}

create(): void {
    this.add.image(400, 200, "logo");
    this.message = this.add.text(400, 380, "Press SPACE", { ... });
    this.message.setOrigin(0.5);
}
```

You never draw. Change a game object; Phaser draws it next frame.

12 / 19

Coordinates and origins



13 / 19

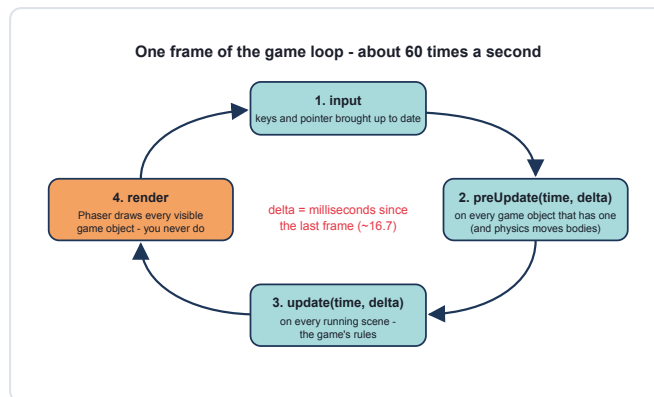
Try it: Hello Phaser

- open `ch01_hello_phaser` , build, serve, open `game.webview`
- click the game, press SPACE
- find the line that picks the colour - and the one that counts

```
this.input.keyboard!.on("keydown-SPACE", () => {  
    this.changeColour();  
});
```

14 / 19

The game loop



15 / 19

A game object that moves itself

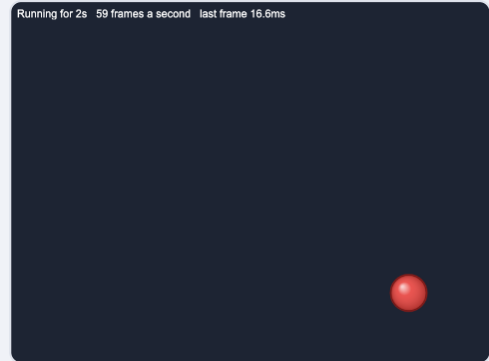
```
export class Ball extends Phaser.GameObjects.Image {
  constructor(scene: Phaser.Scene, x: number, y: number,
    speedX: number, speedY: number) {
    super(scene, x, y, BALL_KEY);
    ...
    scene.add.existing(this); // put it in the scene
  }

  preUpdate(_time: number, delta: number): void {
    const seconds = delta / 1000;
    this.x = this.x + this.speedX * seconds;
    this.y = this.y + this.speedY * seconds;
  }
}
```

16 / 19

Why pixels per second?

- `delta` = milliseconds since the last frame (~16.7 at 60 fps)
- "4 pixels a frame" = 240/s at 60 Hz ... and **576/s** at 144 Hz
- speed x time = distance: the same game on every screen



17 / 19

Summary

- `config` -> `game` -> `scenes` -> `game objects`
- Phaser calls `preload`, `create`, `update`; you never draw
- `setOrigin` says which point (x, y) means; y grows downwards
- events (`keyboard.on(...)`) for one-off actions
- the loop runs ~60 times a second; move by `speed * delta / 1000`

18 / 19

Challenges

1. **Make it yours** - title, colours, text style
2. **Corner labels** - coordinates in each corner, using origins
3. **Reset key** - R puts everything back
4. **More balls** - five random balls, one loop
5. **Bounce counter** - the ball counts, the scene shows
6. **Steer the ball** - arrows push the ball, up to a top speed

Next: **Chapter 2 - a three-scene game**